

Does an airborne thermal detector know when it is wrong?

Confidence calibration of a thermal person-detector under field degradation

Raunak Mondal • HIT-UAV (airborne infrared) • YOLOv8n and YOLOv8s fine-tunes • reproduced end-to-end on Modal

A drone’s detector is only safe to act on if what it reports tracks what it can see. This fine-tunes a YOLOv8n person-detector on HIT-UAV thermal imagery (mAP@0.5 **92.2** clean), degrades the test frame the way a sensor loses a target, and watches **true quality** (mAP) against **reported quality** (box confidence, scored with detection ECE). The control *background_marks* alters only non-target pixels, so anything it moves is the detector reacting to background.

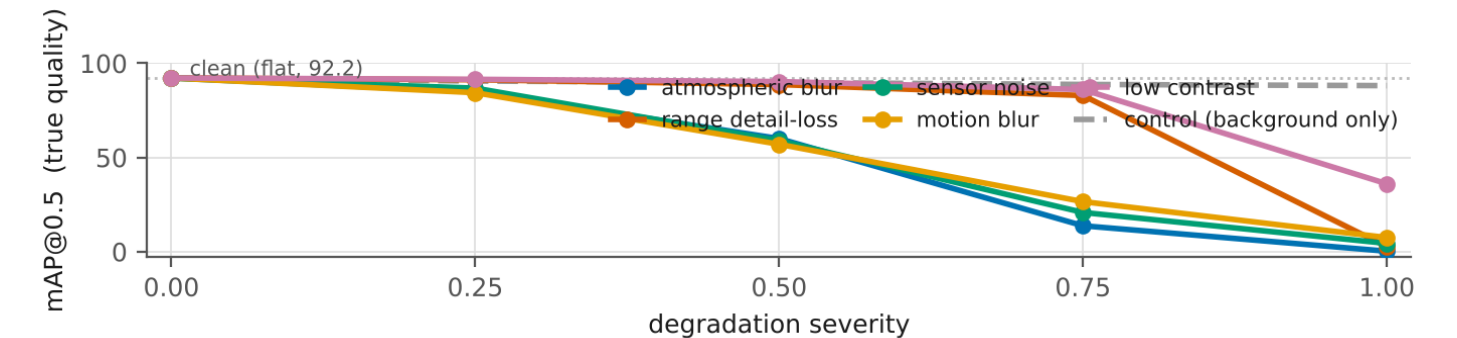


Figure 1: True quality collapses under every field degradation. The control (gray dashed) moves mAP only 4.1 points across its sweep, the noise floor every real degradation clears more than tenfold. Range detail-loss and low contrast hold then cliff; atmospheric blur, sensor noise and motion blur decline steadily.

It fails by going blind, not by lying. The mechanism is recall collapse, not confident errors. Under the three harshest degradations the detector stops detecting: recall falls from 96% to under 6%, detections from 2458 to as few as 20. mAP collapses because targets are missed, not because the surviving boxes are wrong; the survivors are mostly *more* precise than on clean.

at full severity	dets	recall	prec.	conf.	D-ECE [95% CI]
clean	2458	96%	51%	40%	0.17 [0.16, 0.18]
atmospheric blur	20	0.5%	35%	14%	0.25 [0.10, 0.50] (20 dets: unreliable)
range detail-loss	77	3.7%	64%	19%	0.47 [0.35, 0.58]
sensor noise	120	5.9%	64%	17%	0.49 [0.40, 0.58]
motion blur	1002	19%	25%	28%	0.09 [0.07, 0.13] (stays calibrated)
low contrast	687	40%	76%	26%	0.49 [0.45, 0.54]
control (background)	2517	93%	49%	37%	0.15 [0.14, 0.16]

The larger backbone fails identically. The whole sweep on YOLOv8s, 3× the parameters, reproduces every part of this: same clean mAP, same collapse (mAP to 1.5–9.6 on the four hard degradations), D-ECE to 0.77 with the same motion-blur exception, same flat control. The bigger model does not even beat the smaller on clean. It is a property of the detector, not the floor model.

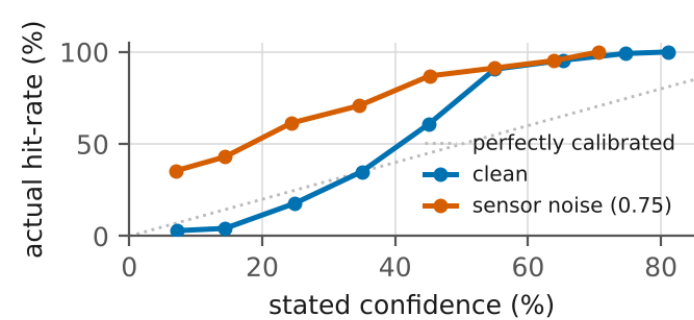


Figure 2: Under degradation the confidence stops being calibrated: the sensor-noise curve lifts far above the diagonal, so a box stated at 7% confidence is right about 36% of the time. It falls but does not report the collapse.

What this means. A fielded thermal detector degrades by going silent, not by lying loudly. Its confidence falls and becomes badly (under-)calibrated, so the confidence stream alone will not tell you the sensor has gone bad or how many targets you are missing; the stack needs a degradation or out-of-distribution signal of its own. The confidence *order* still triages what you do detect.

Honest limits. Perturbations are photometric, so range is simulated as detail-loss at fixed box coordinates rather than a geometric rescale. Single class (person), one thermal dataset. CIs are a 1000-sample frame-level bootstrap on the n run; mAP and recall are point estimates, since missed targets are not in the per-detection log.

Reproduce. modal run modal_app.py::main on HIT-UAV (Kiuyha HF repo); 288-frame test split. Reported-vs-true-quality with a dead control, after SkyDiscover and CRN-Bench.

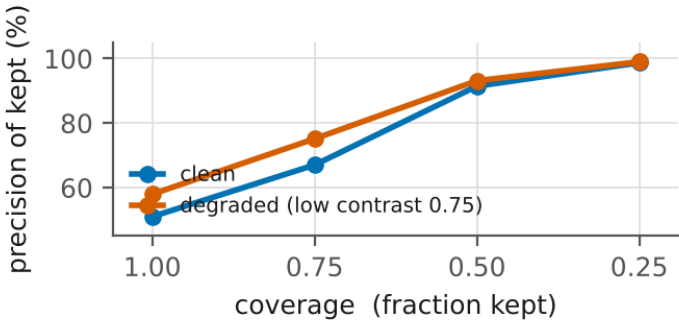


Figure 3: The ranking survives where the calibration does not: keeping the top quarter by confidence holds precision near 99% on clean and under degradation alike. This certifies the detections you keep, not the targets already missed.

Appendix: can post-hoc calibration fix it?

Temperature scaling, the standard confidence-calibration fix, on the YOLOv8n run

If the confidence is miscalibrated under degradation, the obvious question is whether the textbook post-hoc fix repairs it. Temperature scaling learns a single scalar that rescales the confidence, fit on the data you have at calibration time, which in the field means clean, in-distribution frames. Fit on clean here it works where it should: clean D-ECE falls from 0.17 to 0.11. The temperature it learns is **0.45**, a sharpening, because the clean detector is mildly under-confident.

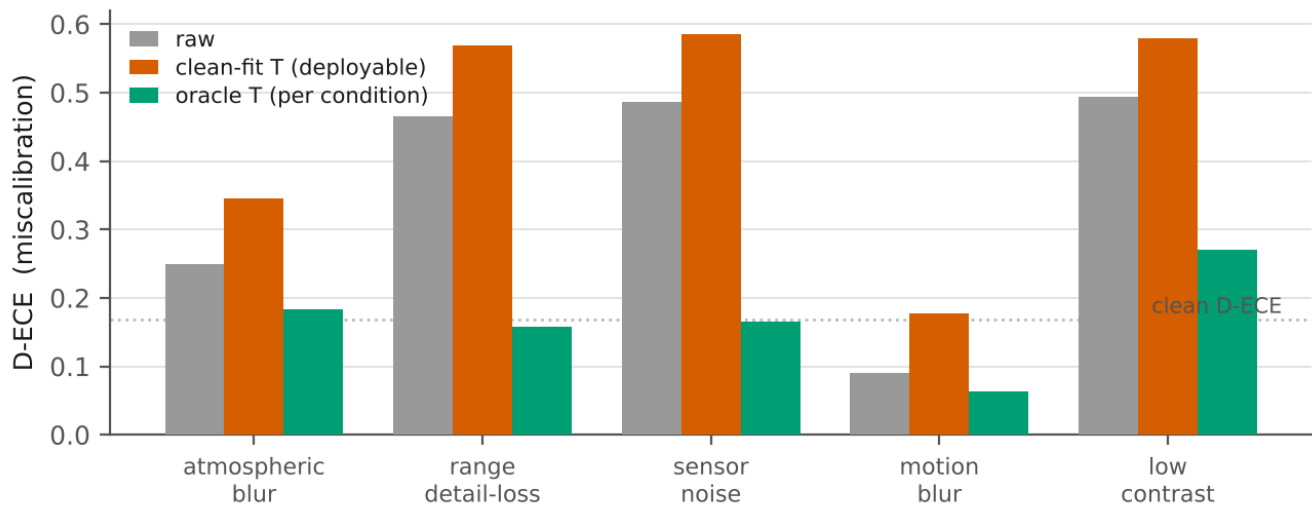


Figure 4: Per degradation at full severity: raw D-ECE, the same metric after the clean-fit temperature (the fix you could actually deploy), and after an oracle temperature fit on each degraded condition (needs labels and knowledge of the degradation, so unavailable in the field). The realistic fix is taller than raw in every case. The dotted line is the clean D-ECE.

Applied to degraded frames, that same clean-fit scaling makes calibration *worse*, by 26% on average, not better. The reason is the per-condition oracle temperatures: the values that would actually fix each degraded condition sit up near 0.20, a heavy softening, the opposite direction from clean's 0.45. Clean wants to sharpen its confidence; degraded wants to soften it. No single global temperature can serve both, so the realistic fix pushes the wrong way out of distribution. And the oracle itself only halves the degraded D-ECE even with everything handed to it.

This reinforces the main result rather than softening it. The miscalibration is not a fixed offset a recalibration layer can absorb; it is a property of the input distribution, and it reverses direction once the sensor degrades. You cannot calibrate your way out of it from in-distribution data. A fielded system needs a signal that the input has degraded, separate from the confidence it is trying to trust.

Reproduce. `python temp_scaling.py` on the `n run's records.csv`: fit one temperature on clean by NLL, apply everywhere, score D-ECE; oracle fits per condition. Temperature scaling is a monotonic rescaling of confidence, so it cannot change the ranking in Figure 3.